

Unit-1 Object Oriented Programming (OOP) Concepts

- Structured programming vs. Object-oriented programming
- Basic OOP concepts: objects, classes, encapsulation, data hiding, inheritance, polymorphism
- Structure of a class
- Creating classes with data-members and member functions
- Private, protected and public members

Structure of C++ Program

FUNCTION BODY	1	<code>#include <iostream></code>	Header File
	2	<code>using namespace std;</code>	Standard Namespace
	3	<code>int main()</code>	Main Function
	4	<code>{</code>	
	5	<code>int num1 = 24;</code> <code>int num2 = 34;</code>	Declaration of Variable
	6	<code>int result = num1 + num2;</code>	Expressions
	7	<code>cout << result << endl;</code>	Output
	8	<code>return 0;</code>	Return Statement
	9	<code>}</code>	

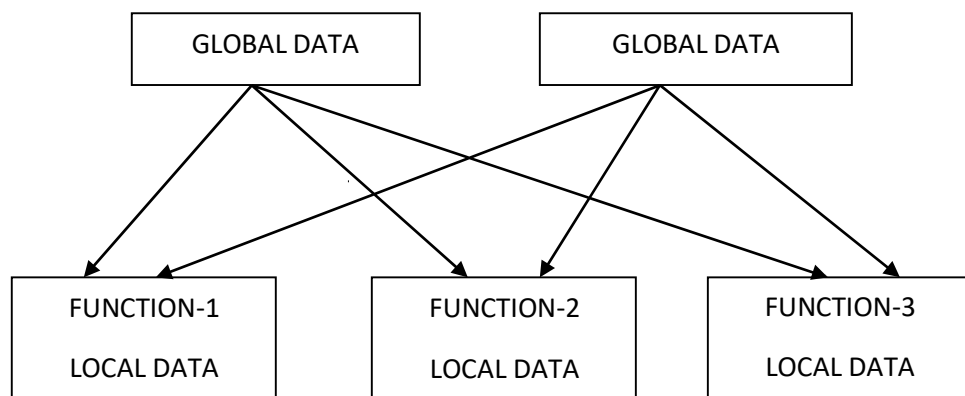
1 Structured programming vs. object oriented programming

1.1 Structured programming was a powerful tool that enabled programmers to write complex programs easily. However, the programs became larger so the structured approach failed to make desired results in terms of bug (error) free, easy-to-maintain and reusable programs.

1.2 Procedure oriented programming:

- Conventional programming using high level languages such as COBOL, FORTRAN and c is commonly known as pop. In the pop approach, the problem is viewed as a sequence of things to be done such as reading, calculating and printing.
- A number of functions are written to accomplish these tasks.
- The primary focus is on functions.
- Pop basically consists of writing a list of instructions or actions for the computer to follow and these instructions are organized into groups known as functions.
- While we concentrate on the development of functions very little attention is given to the data that are being used by the different functions.
- In a multifunction program many important data items are placed as global so that they may be accessed by all the functions. Also each function may have its own local data.

Relationship of data & functions in pop



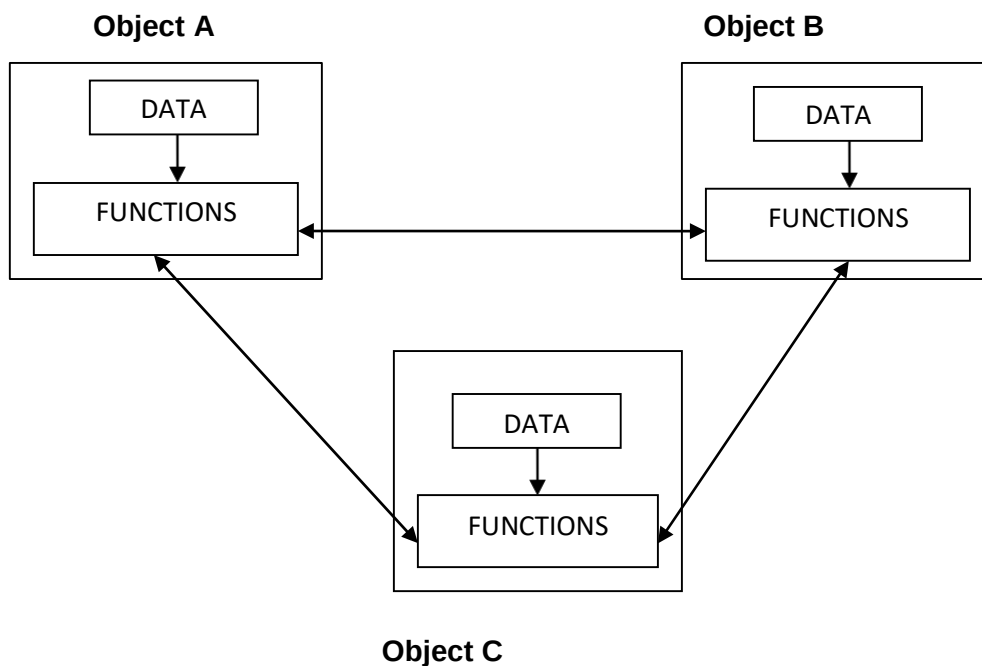
- In a large program it is very difficult to identify what data is used by which function.
- So, the global data are more helpless to an unintentionally change by a function.
- Another drawback is that it does not model real world problems very well.

Characteristics of pop:-

1. Give importance on doing things (algorithms)
2. Large programs are divided into smaller programs known as functions.
3. Most of functions share global data.
4. Data move openly around the system from function to function.
5. Functions transform data from one form to another.
6. Employs top-down approach in program design.

1.3 Object-Oriented Programming Paradigm:-

1. The basic reason of inventing object-oriented approach is to remove some drawbacks encountered in pop.
2. OOP treats data as critical element in the program development and doesn't allow data to flow freely around the system.
3. It ties (binds) the data more closely to the functions that are operated on it and protects data from accidental modification from outside functions.
4. OOP allows decomposition of a problem into a number of entities called objects and then builds data and functions around these objects.
5. The data of an object can be accessed by only the functions associated with the object.



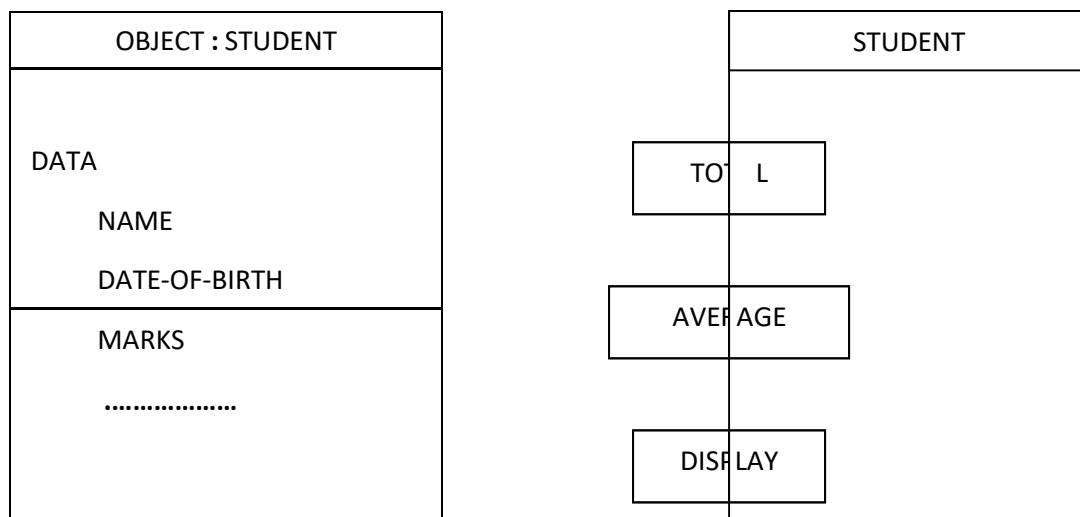
Features of object oriented programming:-

1. Emphasis on data rather than procedure.
2. Programs are divided into objects.
3. Data structures are designed such that they characterized the objects.
4. Functions that operate on the data of an object are tied together in the data structure.
5. Data is hidden and cannot be accessed by external functions.
6. Objects may communicate with each other through functions.
7. Now data and functions can be easily added whenever necessary.
8. It follows bottom-up approach in program design.

We **define** “object-oriented programming an approach that provides a way of modularizing programs by creating partitioned memory area for both data and functions that can be used as templates for creating copies of such modules on demand”.

2.1 Objects:-

- Objects are the basic run-time entities in an object-oriented system.
- They may represent a person, a place, a bank account, and a table of data or any item that the program has to handle.
- Programming problem is analysed in terms of objects and the nature of communication between them.
- Program objects should be chose such that they match closely with the real world objects.
- For example, we can say that individual student is individual object.



Two ways of representing an object

2.2 Classes :-

- It is set of entity like the student is class and individual student is object.
- We know that objects contain data and code to manipulate that data.
- The entire set of data and code of an object can be made a user-defined data type with the help of a class.
- Objects are variables of the type class.
- Once a class has been defined, we can create any number of objects belonging to that class.
- A class is a collection of objects of similar type.

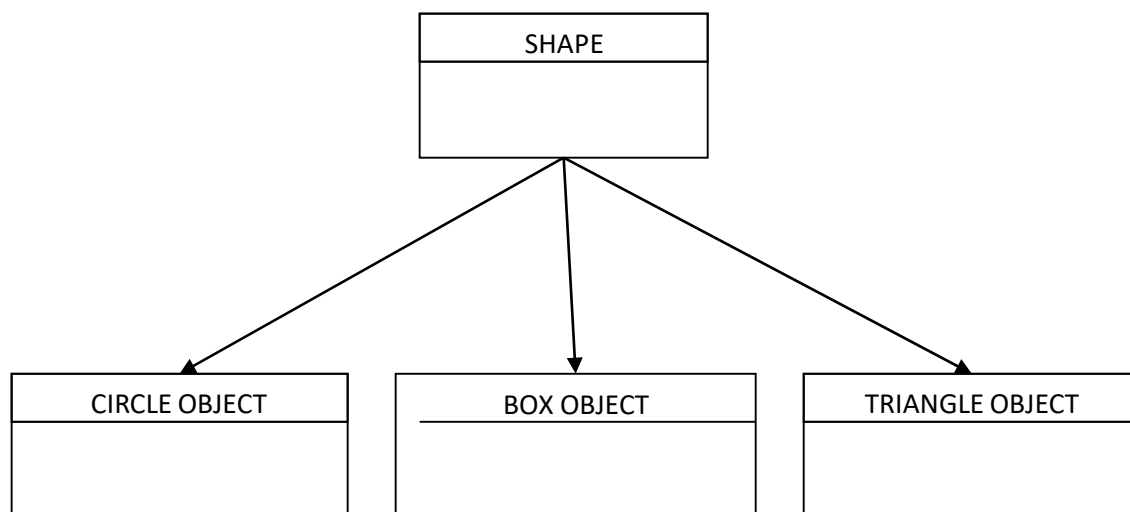
- Example: - mango, apple & orange are members of the class fruit.
- The syntax used to create an object is no different than the syntax used to create an integer object in c.
- If fruit is defined as a class, then the statement is fruit mango, apple, orange;

2.3 Data abstraction and encapsulation:-

- The wrapping up of data and functions into a single unit (called class) is known as encapsulation.
- Data encapsulation is the most striking feature of the class.
- The data is not accessible to the outside world and only those functions which are enclosed in the class can access it.
- These functions provide the interface between the object's data and the program.
- This insulation of the data from direct access by the program is called data hiding or information hiding.
- Abstraction refers to the representation of important features without including the background details or explanations.
- Classes use the concept of abstraction and are defined as a list of abstract attributes such as size, weight and cost and functions to operate on these attributes.
- They encapsulate (bind) all the essential properties (attributes) of the objects that are to be created.
- These attributes are called data members.
- The functions that operate on these data are called methods or member functions.

2.4 Polymorphism :-

- Polymorphism means the ability to take more than one form.
- An operation may execute in different behavior in different instances.
- The behavior depends upon the types of data used in the operation.
- Consider the operation of addition which generates a sum of two numbers.
- If we pass strings as operands then this function concatenates these two strings.
- The process of making an operator (addition, subtraction etc.) To exhibit different behaviors in different instances is known as operator overloading.
- If the function name is same but they have different number of argument or/ and different types (data types) of arguments, then this type is called function overloading.



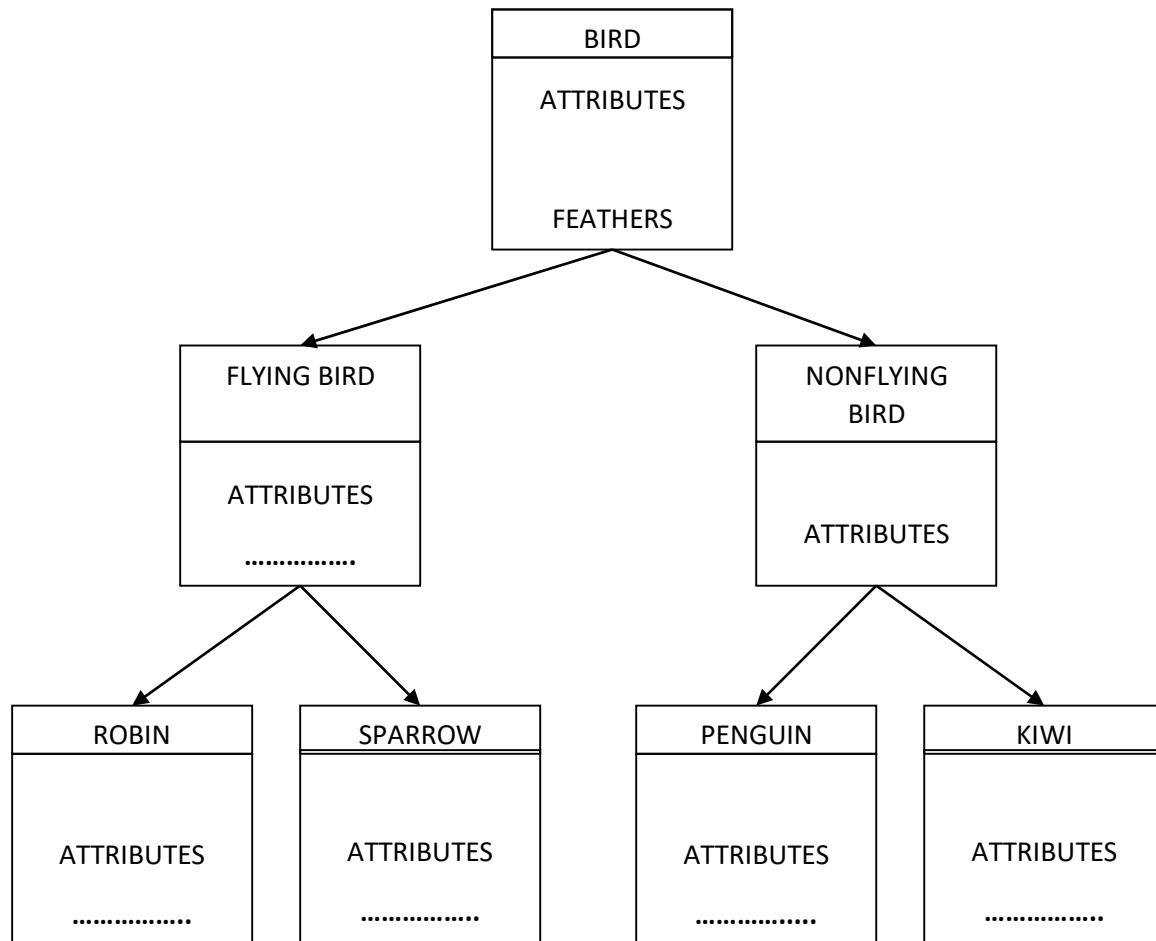
Polymorphism

2.5 Inheritance :

- Inheritance is the process by which objects of one class get the properties of objects of other class.
- It supports the concept of hierarchical classification.
- In OOP, the concept of inheritance provides the idea of reusability.
- This means that we can add additional feature to an existing class without modifying it.

COURSE: SY CA & IT SEMESTER-III UNIT- 1 Object Oriented Programming Concepts

- This is possible by deriving a new class from the existing one.
- The new class will have all the combined features of both the classes.
- The following diagram & example describe that how inheritance is done.



Property inheritance

- In above diagram the main class is bird, the flying bird and non-flying bird are derived from bird class.
- So, both have attributes of bird class.
- Bird robin and sparrow can fly, so both are derived from flying bird class.

2.6 Dynamic binding :-

- Binding refers to the linking of a procedure call with the code to be executed as the response.

- Dynamic binding (also known as late binding) means that the code associated with a given procedure call at run-time.
- It is associated with polymorphism and inheritance.
- This means when a function called the appropriate arguments (numbers & types) are matched.
- This process is done at run-time so it is called dynamic binding.

2.7 Message passing:

- An object-oriented program consists of a set of objects that communicate with each other.
- The process of programming in an object-oriented language, involves the following basic steps.
 1. Creating classes that define objects and their behavior.
 2. Creating objects from class definitions, and
 3. Establishing communication among objects.
- Objects communicate with one another by sending and receiving information.
- The concept of message passing makes it easier to talk about building systems that directly model or simulate their real-world counterpart.
- A message for an object is a request for execution of a procedure, and so it invokes or calls a function (procedure) as the receiving object which generate the desired result.
- Message passing involves the name of the object, the function (message) name, and the information to be sent.

Example:-

```
employee e1;  
class name ↙           ↘ object  
e1.salary (name);  
object ↙       ↓       ↘ information (arguments)  
function name (message)
```

Structure of a class:-

General form of a class definition in C++:

```
class className {  
    private :  
        variable declarations;  
        function declarations;  
    public:  
        variable declarations;  
        function declaration;  
    protected:  
        variable declarations;  
        function declaration;  
};
```

- class, private, public and protected are C++ keywords
- **className** is a valid C++ identifier that indicate name of the class
- variable declarations are **data members** that define properties of class
- function declarations are **member functions** that define behaviour of class
- variables and functions declared within the class are collectively known as **members of class**

Classes:

- A class is a way to bind the data and its associated functions together. It allows the data (and functions) to be hidden, if necessary from external use.
- Defining a class means creating a new abstract data type that can be treated like any other built – in data type.
- A class specification has two parts:
 1. Class declaration.
 2. Class function definitions.

5.1.1 Class declaration:

- It describes the type and scope of its members.

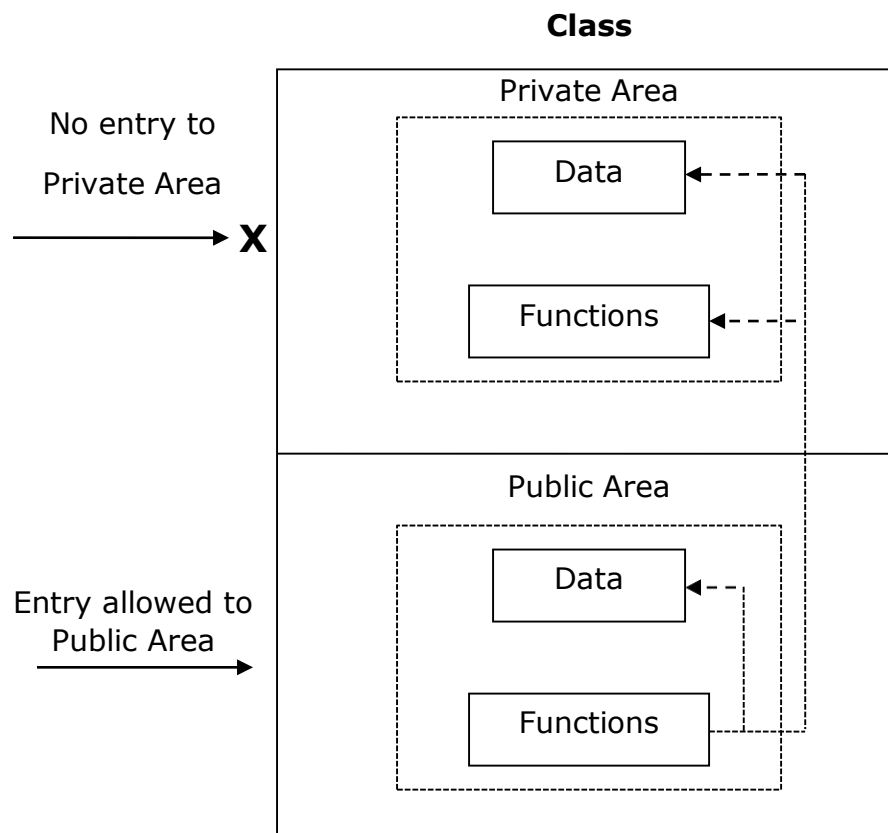
5.1.2 Class function definitions:

- It describes how the class functions are implemented.
- General form of class declaration:

```
class class_name
{
    private:
        variable declarations;
        function declarations;

    public:
        variable declarations;
        function declarations;
};
```

- The keyword class specifies that what follows is an abstract data of type class name.
- The body of a class is enclosed within braces and terminated by a semicolon.
- The class body contains the declaration of variables and functions. These functions and variables are collectively called class members.
- They are usually grouped under two sections, namely, private and public. The keywords private and public are known as visibility labels. These keywords are followed by a colon.
- The class members that have been declared as private can be accessed only from within the class.
- The variable declared inside the class are known as data members and the functions are known as member functions.
- The public members (both functions and data) can be accessed from outside the class.
- The binding of data and functions together into a single class type variable is referred to as encapsulation.



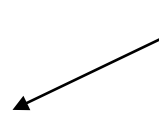
Creating Objects:

- Once a class has been declared, we can create variables of that type by using the class name. in C++ the class variables are known as objects. We may declare more than one object in one statement.

i. e.,

Class name

item x, y, z;



- The necessary memory space is allocated to an object at this stage. Objects can also be created when a class is defined by placing their names immediately after the closing brace, as we do in the case of structures.

- i.e.,

class item

{

.....

.....

.....

}x, y, z;

Accessing Class Members:

- The private data of a class can be accessed only through the member functions of that class. The main() cannot contain statements that access data members.
- Format of calling a member function:

object-name . function-name (actual-arguments);

- A member function can be invoked only by using an object of the same class.

Defining Member Functions:

- Member functions can be defined in two places.
 - Outside the class definition.
 - Inside the class definition.

Outside the class definition:

- Member functions that are declared inside a class have to be defined separately outside the class.
- An important difference between a member function and a normal function is that a member function incorporates a membership “Identity Label” in the header. This “Label” tells the compiler which class the function belongs to.

COURSE: SY CA & IT SEMESTER-III UNIT- 1 Object Oriented Programming Concepts

- **General form of member function definition is:**

```
return-type class-name :: function-name(argument declaration)
{
    Function body
}
```

- **The member functions have some special characteristics as bellow:**

- Several different classes can use the same function name. The “membership label” (class-name) will resolve their scope.
- Member functions can access the private data of the class. A non-member function cannot do so, but the friend function can access private data.
- A member function can call another member function directly, without using the dot operator.

Inside the class Definition:

- Another method of defining a member function is to replace the function declaration by the actual function definition inside the class.

- Example:

```
class item
{
    private:
        int number;
        float cost;

    public:
        void getdata();                //Declaration.

        //Inline function
        void putdata()
        {
            cout<<"Number :"<<number<<endl;
            cout<<"Cost :"<<cost<<endl;
        }
};

void item :: getdata()
```

```
{  
    cout<<"\nEnter Number :";  
    cin>>number;  
    cout<<"Enter Cost : ";  
    cin>>cost;  
}  
void main()  
{  
    item i1, i2;  
    i1.getdata();  
    i1.putdata();  
}
```

- When a function is defined inside a class, it is treated as an inline function. Therefore all the restrictions and limitations that apply to an inline function are also applicable here.

Normally, only small functions are defined inside the class definition.

Access specifiers, implementing and accessing class members

- Each user has different access privileges to the object. A class differentiate between access privileges by partitioning its contents and associating each one of them with any one of the following keywords:
 - Private
 - Protected
 - public
- These keywords are called access control Specifiers. All the members that follow a keyword (up-to another keyword) belong to that type.
- If no keyword is specified, then the members are assumed have private privilege. The following table specifies control of privileges.

Access Specifier	Accessible to			
	Own class member	Derived class member	Other class member	Object of a class
Private	Yes	No	No	No
Protected	Yes	Yes	No	No
Public	Yes	Yes	Yes	Yes

7.1 Private Member :

- The private members of a class have strict access control. Only the member functions of the same class can access these members.
- The private members of a class are inaccessible outside the class, thus, providing a mechanism for preventing accidental modifications of the data members.

```

Class person
{
    Private :
    // private members
    .....
    Int age ;
    Int getage();
    .....
};
Person p1;

```

```

A=p1.age;           // cannot access private data
P1.getdata();       // cannot access private data

```


COURSE: SY CA & IT SEMESTER-III UNIT- 1 Object Oriented Programming Concepts

- The following example illustrates the situation when all the members of a class are declared as private:

Class inaccessible

```
{  
    int x ;  
    void display();  
    {  
        cout<<"\n data ="<<x;  
    }  
};  
void main()  
{  
    inaccessible obj1;  
    obj1.x=5;           // error invalid access  
    obj1.display();     // error invalid access  
}
```

- The class having all the members with private access control is of no use; there is no means available to communicate with the external world. Therefore, classes of the above type will not contribute anything to the program.

7.2 Protected Member :

- The access control of the protected members is similar to that of private members and has more significance in inheritance. The member function of the derived class can access the protected data member.
- Only class member and derived class member can access protected data, other class member cannot access.

class person

COURSE: SY CA & IT SEMESTER-III UNIT- 1 Object Oriented Programming Concepts

```
{
    protected : //access specifier    // protected members
    .....
    int age ;
    int getage();
    .....
};
person p1;
int a =p1.age;           //same as private can not access protected member
p1.getdata();           //same as private can not access protected member
```

public members

- The members of class , which are to be visible (accessible) outside the class, should be declared in public section.
- All data members and functions declared in the public section of the class can be accessed without any restriction from anywhere in the program, either by function that belongs to the class or by those external to the class.
- Accessibility control of public members is shown in figure.

```
class person
{
    public : //access specifier
            // public members
    .....
    int age ; public data
    int getage(); public function
    .....
};
person p1;
int a=p1.age;           //class access public data
p1.getdata();           // can access public function
```